Portfolio Optimization on Classical and Quantum Computers Using PortFawn

Moein Owhadi-Kareshk
Department of Computing Science
University of Alberta
Edmonton, Canada
owhadika@ualberta.ca

Pierre Boulanger
Department of Computing Science
University of Alberta
Edmonton, Canada
pierreb@ualberta.ca

arXiv:2112.08998v1 [cs.CE] 16 Dec 2021

***Abstract*—Portfolio diversification is one of the most effective ways to minimize investment risk. Individuals and fund managers aim to create a portfolio of assets that not only have high returns but are also uncorrelated. This goal can be achieved by comparing the historical performance, fundamentals, predictions, news sentiment, and many other parameters that can affect the portfolio's value. One of the most well-known approaches to manage/optimize portfolios is the well-known mean-variance (Markowitz) portfolio. The algorithm's inputs are the expected returns and risks (volatility), and its output is the optimized weights for each asset in the target portfolio. Simplified unrealistic assumptions and constraints were used in its original version preventing its use in practical cases. One solution to improve its usability is by altering the parameters and constraints to match investment goals and requirements. This paper introduces `PortFawn` , an open-source Python library to create and backtest mean-variance portfolios. `PortFawn` provides simple-to-use APIs to create and evaluate mean-variance optimization algorithms using classical computing (real-valued asset weights) as well as quantum annealing computing (binary asset weights). This tool has many parameters to customize the target portfolios according to the investment goals. The paper introduces the background and limitations of the mean-variance portfolio optimization algorithm, its architecture, and a description of the functionalities of `PortFawn` . We also show how one can use this tool in practice using a simple investment scenario.**

***Index Terms*—Investment `PortFawn` , Portfolio Optimization, Python Library, Quadratic Optimization .**

I. Introduction

There is currently a vast range of tools for investors to help invest in equities, fixed-income, real estates, commodities, futures, etc. Each of these investment tools has its characteristics, such as risks and returns. Investors usually choose one or multiple of them to make an investment based on their financial needs. It is shown that investing in assets with uncorrelated returns might help at decreasing risks while only slightly sacrificing on returns [1]. Investors, thus, create a portfolio that is a basket of different assets to achieve their financial goals while eliminating unnecessary risks.

There are two main types of risks for each asset, systematic and non-systematic risks [2]–[5]. The systematic risks are common to all or most assets, while the non-systematic risk only corresponds to a specific asset. As an example of systematic risk, if the Gross Domestic Product (GDP) of a country dramatically decreases due to poor monetary policies, all of the assets listed in the markets will be impacted. This is systematic risk, and investors who invest in such a country cannot decrease the risk. However, a decrease in the return of just one company in the market due to bad business decisions is an example of non-systematic risk, and investors can decrease its impact on their portfolio by diversification. A good portfolio aims at holding assets with high returns that are diverse and uncorrelated.

In this paper, we focus on the mean-variance portfolio optimization approach, which is based on the Markowitz theory [1]. This approach is based on a sound theoretical background that uses the expected (mean) returns and risks (the covariance matrix of asset returns) to minimize the risk of a portfolio for a given target return, or equivalently, maximize the portfolio return for a target risk level. This approach defines portfolio optimization as a quadratic (or linear) programming problem with a set of linear (or quadratic) constraints. Summarizing the returns data to the expected returns and the covariance matrix, and eliminating other available information about assets can be seen as an oversimplification. However, this theory is effective enough in some scenarios and can be used as a complementary approach to other investment approaches.

The mean-variance portfolio is vastly studied in the literature, including studies on employing different optimization methods such as linear models [6] and evolutionary optimization methods [7], [8]. The mean-variance portfolio

`PortFawn` is an open-source Python package under an MIT license to provide easy-to-use and well-tested tools to researchers. `PortFawn` should solely be used for educational and research purposes and not as an investment analysis tool. Individuals who seek financial and investment data, information, visualizations, and advice are highly encouraged to seek personalized advice from a certified financial advisor. The authors do not accept any responsibility and/or liability for the consequences or any damages resulting in using this tool for any purposes other than research and education. We use the D-Wave quantum computer (*Developer* account) and Ocean software infrastructure as part of our evaluations.

optimization can be expressed as either single [9]–[11] or multi-objective optimization problem [12]–[19]. Adding additional constraints to the original mean-variance portfolio optimization algorithm is also investigated in the literature to make the assumptions and constraints more realistic [20]–[26].

This paper introduces `PortFawn` , an open-source Python library that we have developed to implement and backtest the mean-variance portfolio optimization approach with options to change the optimization goal, constraints, and other portfolio parameters. This tool (1) collects the asset prices for a given list of assets, (2) calculates the expected returns and risks, (3) builds a portfolio, and (4) performs backtesting to evaluate the optimized portfolio. `PortFawn` supports multiple cost functions and constraints where each can be used under certain assumptions. Besides the ability to run the optimization on classical computers, `PortFawn` also solves the portfolio optimization problem using quantum annealing computers.

This is a meta-heuristic optimization approach that leverages the power of quantum computers to optimize a given objective function with binary variables [27]–[29]. This approach has a close relation to its classical version, simulated annealing [30]–[32]. Quantum annealing is based on adiabatic quantum computing capable of solving Quadratic Unconstrained Binary Optimization (QUBO). The solver aims to minimize the energy Hamiltonian (function) of the optimization problem by slowly applying the problem Hamiltonian to an initial quantum state [33]–[35]. Optimization by quantum annealing has been successfully applied to a vast range of problems in the literature such as maximum-cut [36], [37], graph coloring [38]–[40], graph partitioning [41], [42], independent set [43], knapsack [44], satisfactory [45]–[47], and machine learning [48]–[50] problems.

Quantum annealing is also used to solve the portfolio optimization problem [51]–[53] before. Still, most of the literature only focuses on quantum computers' internal abstract performance and does not study the implication of using it in portfolio optimization. Moreover, developing a new problem formulation method needs individual efforts by researchers to develop the code for optimizing portfolios and backtesting them. `PortFawn` provides a set of unified APIs to implement and evaluate any novel idea in this domain with minimum effort.

`PortFawn` logs every step in building and backtesting portfolios to make the result validation and verification easier. It also contains unit and integration tests to ensure that the tool does not break after a change. We release the `PortFawn` code with its documentation as an open-source tool under the MIT License[1]. We encourage the reader to read the online documentation for any recent changes and use this paper as a first step of understanding the background of the mean-variance portfolio optimization and general information about `PortFawn`. We also invite the reader to contribute to the code and the documentation of this work.

[1]https://github.com/mkareshk/PortFawn

We designed a portfolio optimization and backtesting scenario and show the sample results to demonstrate our tool's functionality. We use the historical data of nine Exchange-Traded Funds (ETFs) between January 1, 2015 and December 31, 2020 for this backtesting. We use the data of 40 days for creating portfolios and 5 for out-of-sample evaluation using overlapping time windows. The results show that each of the mean-variance portfolio optimizations works according to what we expected. For example, we observe very low volatility in the minimum volatility portfolio and a high return in the maximum return portfolio.

In Section II we explain how the mean-variance portfolio optimization works. We then introduce our `PortFawn`, its architecture, and characteristics in Section III. In Section IV we perform backtesting using `PortFawn` to show the functionality of our tool in practice. We finally discuss the limitations of mean-variance portfolio optimization in Section V.

II. Portfolio Optimization

It is important to understand how the mean-variance portfolio optimization works before explaining the main features of `PortFawn`. In this section, we first review the calculations of returns and risks for assets and portfolios in Section II-A and II-B, respectively. We then review the essential definitions of portfolio optimization in Section II-C. In the end, we explain how to use quantum annealing for portfolio optimization in Section II-D

A. Returns and Risks of Assets

The mean-variance portfolio optimization assumes that asset returns are i.i.d random variables derived from a normal distribution and can uniquely be parametrized by their means and variances. If the price of asset i changes from p_1 to p_2 during an specific period of time, the asset return r_i is calculated as

$$r_i = \frac{p_2 - p_1}{p_1}. \tag{1}$$

In reality, one can use (1) to calculate the returns only at the end of the investment period when both p_1 and p_2 are known. We, therefore, use the *expected returns* in calculations to be able to create a portfolio for the future and analyze its performance. The expected return of an asset is a prediction (an educated guess) about the asset's performance in the future using historical data on price changes and/or fundamental asset information. However, assuming a normal distribution for the returns, using the average of the returns is the best guess and a natural choice as the expected return. If $r_i^1, r_i^2, ..., r_i^k$ are the returns of asset i in k subsequent historical observations, e.g. daily prices, the expected return is calculated by

$$\bar{r}_i = \mu_{r_i} = E[r_i] = \frac{1}{k}\sum_{j=1}^{k} r_i^j. \tag{2}$$

By considering volatility as a risk indicator, the variance is used to characterize the risk of asset i.

$$\sigma_i^2 = E[r_i - \bar{r}_i] = \frac{1}{k-1}\sum_{j=1}^{k}(r_i^j - \bar{r}_i)^2. \tag{3}$$

However, the standard deviation is usually used in analysis since its unit is as same as the returns.

B. Returns and Risks of a Portfolio

Portfolio optimization aims to create an optimal basket of N assets to reduce non-systematic (diversifiable) risks while gaining the highest possible return. Let t_0 to t_1 be the time frame to use its historical data to create a portfolio. We then hold the portfolio from time t_1 to t_2 and then evaluate its performance in this investment configuration. The mean and standard deviation of historical returns between t_0 and t_1 is the expected return and risk for t_1 to t_2. This is a very strong generalization that directly impacts portfolio performance.

We show the asset allocation weights by vector $w = [w_1, w_2, ..., w_N]^T$ where $w_i \in \mathbb{R}$ is the weight of the ith asset in the portfolio such that $0 \leq w_i$ which means that we just allow long positions and avoid short-selling. Furthermore, we aim to invest all the available capital and thus, $w^T 1 = 1$, i.e. $\sum_{i=1}^{N} w_i = 1$. The vector representation of asset returns is $r = [r_1, r_2, ..., r_N]^T$ and Σ is the covariance matrix of the asset returns. The portfolio return and variance are calculated by

$$r_p = w^T r = \sum_{i=1}^{N} w_i r_i, \tag{4}$$

$$\sigma_p^2 = w^T \Sigma w = \sum_{i=1}^{N}\sum_{j=1}^{N} w_i \sigma_{ij} w_j. \tag{5}$$

C. Mean-Variance Portfolio Optimization

Now that we know how to calculate the risk and return of a portfolio, we can start finding the optimal asset allocation w^* for various objectives. In the rest of this subsection, we introduce different approaches to formulate the mean-variance portfolio optimization problem. Each of these approaches aims to find w^* that optimizes a specific objective concerning a set of constraints. Investment requirements and goals determine which approach to choose.

a) Minimum Volatility Portfolio (MVP): The goal of this portfolio is to obtain the desired return R while minimizing the volatility. It also ensures to invest the whole capital and do not perform any short-selling.

$$\begin{aligned} \min_{w} \quad & w^T \Sigma w && \text{(6a)} \\ \text{subject to} \quad & w^T r = R && \text{(6b)} \\ & w1 = 1 && \text{(6c)} \\ & w_i \geq 0, i = 1, ..., N && \text{(6d)} \end{aligned}$$

b) Maximum Return Portfolio (MRP): This approach aims to maximize the portfolio's returns for a given desired volatility V. Similar to MVP, this approach also ensures investing all available capital and does not perform short-selling.

$$\begin{aligned} \max_{w} \quad & w^T r && \text{(7a)} \\ \text{subject to} \quad & w^T \Sigma w = V && \text{(7b)} \\ & w1 = 1 && \text{(7c)} \\ & w_i \geq 0, i = 1, ..., N && \text{(7d)} \end{aligned}$$

c) Maximum Sharpe Ratio Portfolio (MSRP): The Sharpe ratio is one of the risk-adjusted return indicators based on the risk-free asset return r_{rf}.

$$S_p = \frac{r_p - r_{rf}}{\sigma_p}. \tag{8}$$

The risk-free asset is considered a safe investment with zero risks. While no asset can be regarded as entirely safe, a set of them is assumed to be risk-free in quantitative analysis. Treasury bills issued by the U.S. Department of the Treasury [2] is the one that is widely used in the industry.

This approach aims to maximize S_p, while ensuring investing all available capital and using long positions only:

$$\begin{aligned} \max_{w} \quad & S_p && \text{(9a)} \\ \text{subject to} \quad & w1 = 1 && \text{(9b)} \\ & w_i \geq 0, i = 1, ..., N && \text{(9c)} \end{aligned}$$

d) Multi-Objective Portfolio (MOP): This approach aims to optimize both portfolio return and volatility at the same time while ensuring investing all available capital with long positions only. The cost function of this portfolio is the linear combination of return and risk using the risk aversion parameter λ.

$$\begin{aligned} \min_{w} \quad & w^T \Sigma w - \lambda w^T r && \text{(10a)} \\ \text{subject to} \quad & w1 = 1 && \text{(10b)} \\ & w_i \geq 0, i = 1, ..., N && \text{(10c)} \end{aligned}$$

e) Market Capitalization Portfolio (MCP): Contrary to the previous portfolio optimization approaches, this approach does not need to solve an optimization problem. Instead, it assigns weights to each asset according to its market capitalization. Most passive index funds use this approach for asset management.

f) Equal-Weighted Portfolio (EWP): As the name suggests, this portfolio assigns equal weights to all assets regardless of any other factor. If the investment universe is diverse (as in our experiment in this paper), this approach can be pretty practical, bringing simplicity to investment.

[2] https://www.treasury.gov/

D. Portfolio Optimization Using Quantum Annealing

To use quantum annealing, the objective function of the portfolio optimization problem is represented as a QUBO model and the aim is to *select* or *not select* assets to create a long-only portfolio. Therefore, the asset weights are binary, and not real-valued here.

In a QUBO model with N binary variables, each variable is represented by $x_i \in \{0, 1\}$, its linear bias is Q_{ii} and its quadratic bias with an arbitrary variable x_j is Q_{ij}. A QUBO model can be also represented as a graph where $E = \{x_1, x_2, ..., x_N\}$ is the set of nodes and Q is the adjacency matrix. The QUBO model defines the optimization objective as an energy function.

$$E(x) = \sum_{i=1}^{N} Q_{ii} x_i + \sum_{\substack{i,j=1 \\ i<j}}^{N} Q_{ij} x_i x_j. \tag{11}$$

The scalar notation of MOP's objective (10a) is

$$\sum_{i=1}^{N} -\lambda r_i w_i + \sum_{i,j=1}^{N} \sigma_{ij} w_i w_j, \tag{12}$$

which is identical to the QUBO model (11) when $x_i = w_r$, $Q_{ii} = -\lambda r_i$, and $Q_{ij} = \sigma_{ij}$. We use QUBO to program a multi-objective portfolio optimization problem ($\lambda = 1$ for simplicity) on a quantum computer and call it Binary Multi-Objective Portfolio (BMOP). We ensure the two constraints (long-only positions and investing all available capital) by normalizing the wights.

Finding a local minimum (solution), instead of the global minima, is a common challenge in classical solvers. However, quantum solvers have a higher chance of finding the global minimum by utilizing the quantum phenomena called *quantum tunnelling*.

III. The Architecture of PortFawn

This section discusses the most important classes implemented in `PortFawn` .

Historical Market Data: The `MarketData` class collects the historical prices for a given list of assets (tickers) and a time period using the `yfinance`[3] Python package and calculates basic statistics. This class caches the price data to make the external `yfinance` API calls efficient. Asset returns, cumulative returns, correlation and covariance matrices, and return distributions are calculated in this class. This class also has APIs to visualize the market data.

Expected Returns and Risks: The `ExpectedStats` class calculates the expected returns and risks for a given `MarketData` instance. The mean-variance portfolio is susceptible to these two inputs, and therefore, it is important to make these robust as much as possible. Our tool supports three types of calculations for the expected returns and risks, as follows:

- **full:** Simply calculates the mean and covariance of the returns using the whole data.
- **random:** Selects a subset of returns (a time window) randomly and calculates the mean and variance for only that subset, and repeats this process for a given number of times. The results are aggregated at the end using the median.
- **weighted:** Similar to the random option, but assign more weights to the most recent returns.

Portfolio: The `Portfolio` class is one of the essential classes in `PortFawn` , which generates a portfolio for a given list of assets, period, and portfolio objective and constraints. This class also calculates and visualizes the performance metrics such as returns, risks, annualized returns, annualized risks, cumulative returns, and Sharpe ratio for the optimized portfolio. The portfolio optimization objectives currently supported by this class are equal-weighted, minimum risk, maximum return, and maximum Sharpe ratio. This class uses the `PortfolioOptimization` class to perform optimization on classical or quantum computers.

Backtesting: Creating a portfolio for just a one-time window and testing it with a subsequent time window is a naive way of performance evaluation since the effectiveness of an investment strategy might change over time due to the changes in asset's fundamentals or macroeconomic factors. For a meaningful evaluation, we need to create and test portfolios in multiple overlapping rolling windows to ensure the consistency of the results over time. The `BackTest` class performs backtesting in `PortFawn` , and the user can choose the number of training and testing days, the period of backtesting, and all other portfolio parameters.

IV. Illustrative Example of Using PortFawn in Practice

In this section, we demonstrate a show-case for using `PortFawn` . We do not explicitly mention the code to generate figures since the `PortFawn` API might change in the future. Still, the examples with the updated code are available on the documentation page.

A. Backtesting Setup

First, let's define the assumptions and parameters for our backtesting scenario that compares the available portfolio optimization approaches in `PortFawn` . Table I introduces the parameters we choose for our experiment. We choose nine diverse iShares ETFs (listed in the U.S.) in Table II to be used in this experiment.

B. Historical Market Data

We can now review the historical data according to the backtesting setup. We show the cumulative returns of the assets in Figure 1. As we expect, equities have higher returns than fixed incomes in general.

In Figure 2, we illustrate the distribution of daily returns as box plots in symmetric log scale. Confirming the previous

[3]https://pypi.org/project/yfinance/

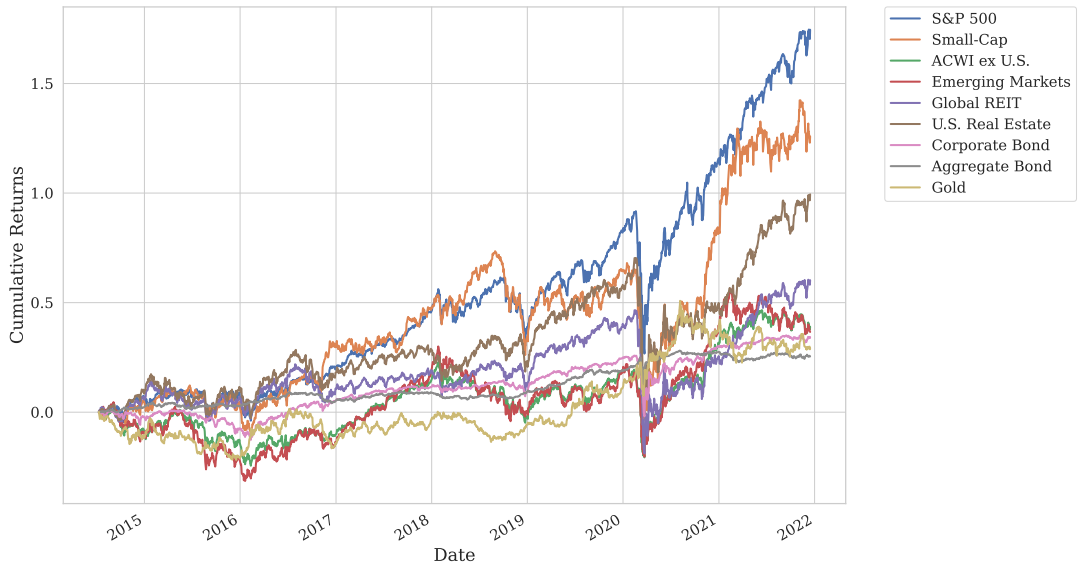

Fig. 1. Cumulative Returns of Assets. Real Estate and U.S. equities have the highest returns in our data, while fixed-income assets were less volatile and with lower returns.

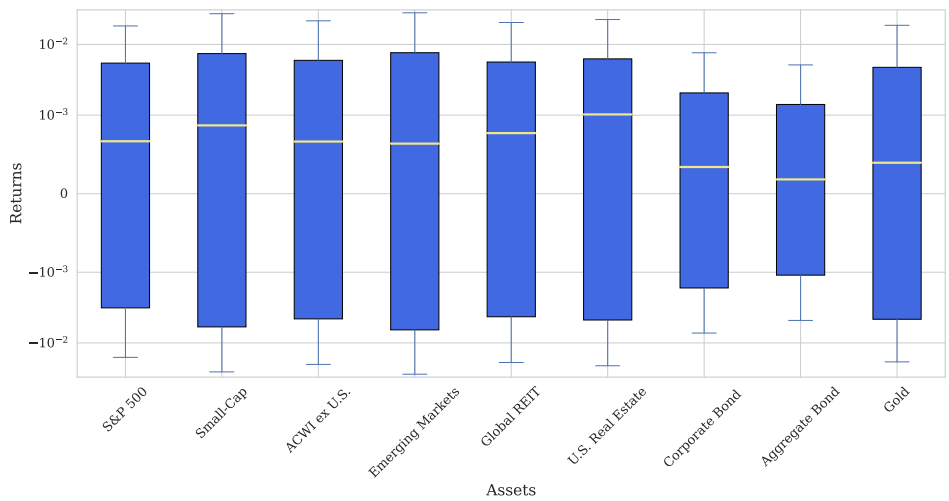

Fig. 2. Distribution of Daily Asset Returns. Equities and gold are more volatile than fixed income. Equities' returns are higher than both gold and fixed income assets.

TABLE I
BACKTESTING PARAMETERS FOR OUR ILLUSTRATIVE EXAMPLE

#	Parameter	Value
1	Backtesting starting date	January 1, 2015
2	Backtesting ending date	December 31, 2020
3	Portfolio training period	40 days
4	Portfolio testing period	5 days
5	Risk-free rate	0.0% (for simplicity)
6	Returns to use	Daily
7	Target return (R in (6b))	20.0%
8	Target volatility (V in (7b))	5.0%
9	Lower Bound Asset Weight	2.0%
10	Upper Bound Asset Weight	98.0%

figure, this figure shows that the risks of equities and gold are more than fixed incomes.

The correlations of daily returns reveal information about how the asset returns change compared to each other during the analysis period. The asset correlation in Figure 3 illustrates that equities, real estate, and corporate bonds are highly correlated, while aggregate bonds and gold are much less correlated with the rest of assets.

C. Portfolios' Performance

We use `PortFawn` to create portfolio with different goals using the list of assets in Table II and configurations in Table I. We first introduce out-of-sample evaluation results.

TABLE II
LIST OF ASSETS USED IN OUR BACKTESTING SCENARIO

#	Ticker	Asset Name	Short Name	Asset Class	Market
1	IVV	iShares Core S&P 500 ETF	S&P 500	Equity	U.S.
2	IJR	iShares Core S&P Small-Cap ETF	Small-Cap	Equity	U.S.
3	ACWX	iShares MSCI ACWI ex U.S. ETF	ACWI	Equity	Global
4	IEMG	iShares Core MSCI Emerging Markets ETF	Emerging	Equity	Emerging
5	REET	iShares Global REIT ETF	Global RS	REIT	Global
6	IYR	iShares U.S. Real Estate ETF	U.S. RS	REIT	U.S.
7	HYG	iShares iBoxx High Yield Corporate Bond ETF	Corp. Bond	Fixed-Income	U.S.
8	AGG	iShares Core U.S. Aggregate Bond ETF	Agg. Bond	Fixed-Income	U.S.
9	IAU	iShares Gold Trust	Gold	Commodity	U.S.

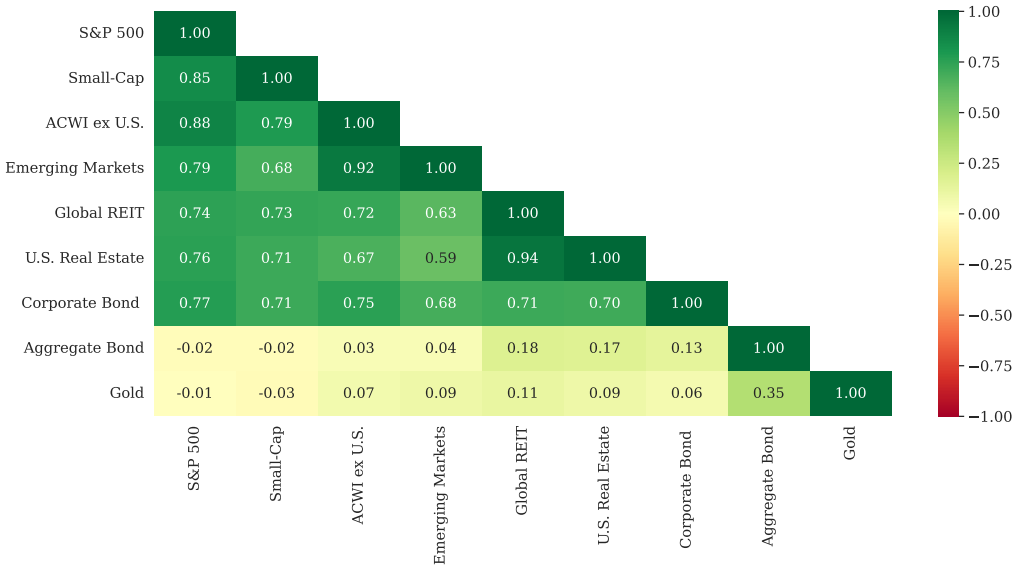

Fig. 3. Correlation of Daily Asset Returns. Aggregate bond an gold have low correlation with the other assets, which makes them proper candidates to bring diversity to portfolio.

In Figure 4, we illustrate the cumulative returns of a baseline portfolio (EWP), three mean-variance portfolios created on a classical computer (MRP, MVP, and MSRP), and a binary-weight portfolio (BMOP) created on a quantum computer. The distributions of daily returns are also demonstrated in Figure 5. As we expect, MRP has the highest returns while MVP is less volatile than the other portfolios, and MSRP seems a balance between MVP and MRP. While BMOP is formulated differently and cannot have fractional asset weights, it works very well which shows potential for using quantum computers in portfolio optimization. Furthermore, EWP seems an interesting one here since both its return and volatility are acceptable while it brings simplicity to the investment.

Figure 6 shows the correlation between different portfolios. We can see a relatively high correlation between different approaches.

To be able to compare the asset and portfolio returns and risks more clearly, we illustrate annualized volatility (standard deviation of returns) and expected (average) returns of assets (in blue) and portfolios (in red) in Figure 7 as an in-sample evaluation. The dashed black line shows the efficient frontier. In general, investing in portfolios reduces volatility while sacrificing returns. Choosing a balance between these two depends on investors' preferences.

V. LIMITATIONS OF THE MEAN-VARIANCE PORTFOLIO

Despite the theoretical power of the mean-variance portfolio, the original form of this approach makes several unrealistic assumptions that limit its usage in practice. These assumptions can be relaxed by altering the cost function and constraints, but we focus only on the original form of the portfolio here.

a) Asset Liquidity: The mean-variance portfolio assumes that all assets in the investment universe are completely liquid, and we can buy and sell them at any time. While the soft version of this assumption holds for large-cap assets in developed markets, it is not always the case for the rest of the assets.

b) Costs: Turn-overs, transactions, and management are costly. However, the mean-variance portfolio does not consider these costs. Such a significant elimination could result in high returns and low risks in backtesting while having a poor performance in practice.

c) Statistical Limitations: The mean-variance portfolio assumes that returns are independent and identically dis-

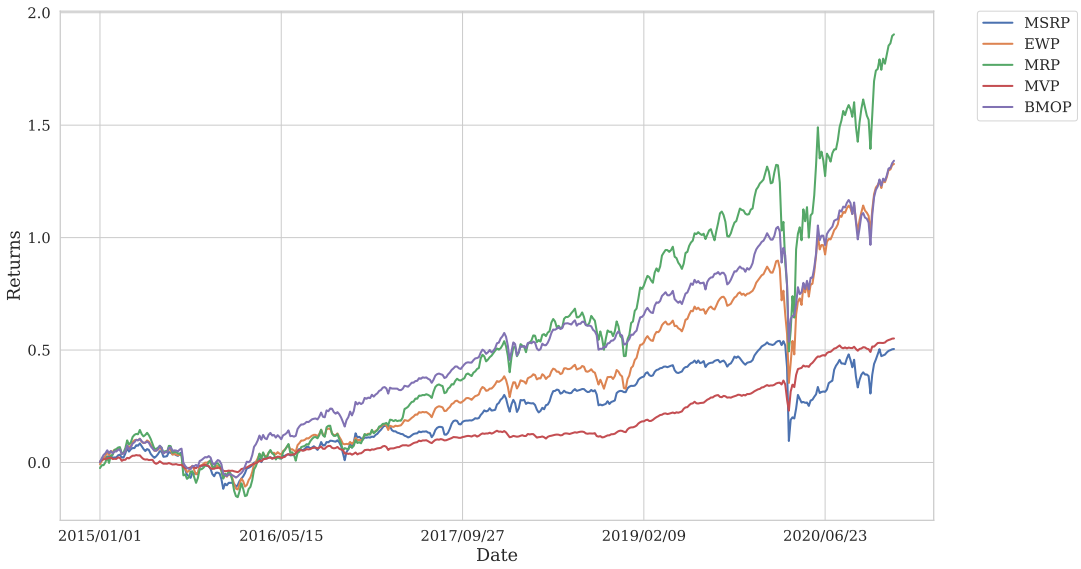

Fig. 4. Cumulative Portfolio Returns. MRP and BMOP have the highest returns.

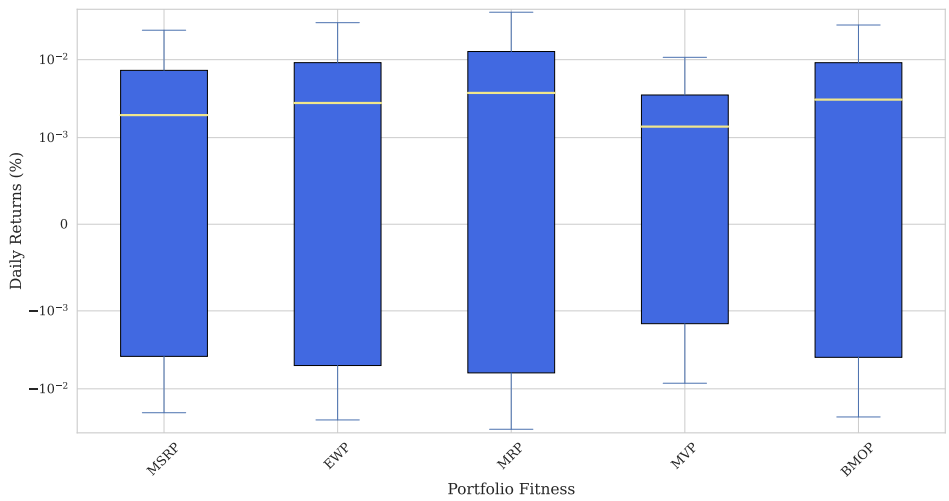

Fig. 5. Distribution of Daily Portfolio Returns.

tributed (i.i.d), and they can be expressed solely by their returns and covariance matrix. Although this assumption could hold for a short period and some assets, its relevance needs to verify for specific historical data. This approach also assumes that the mean and variance of returns are the only factors an investor may aim to optimize and investors have a clear view of their real expectations about the returns they seek and the volatility they can tolerate. Lastly, this approach assumes that the expected returns are a good prediction for future returns, which is not always the case.

d) Dividends: The mean-variance portfolio only considers price changes to calculate returns and eliminate dividends in calculations. This makes calculations unrealistic for high-dividend assets.

e) Asset Concentration: The mean-variance portfolio approach tends to generate portfolios with a high concentration in only a few assets. This might be the local (or even global) optimal asset allocation according to the optimization problem definition, but it is not favourable from an investment viewpoint.

VI. Conclusion

We introduce `PortFawn` in this paper. This open-source Python library is capable of generating different mean-variance portfolio optimization scenarios according to the input parameters. Accepting a vast range of configuration parameters allows the end-user to overcome some of the restrictions and limitations of the original version of this approach to create more realistic portfolios. `PortFawn` performs the whole pipeline

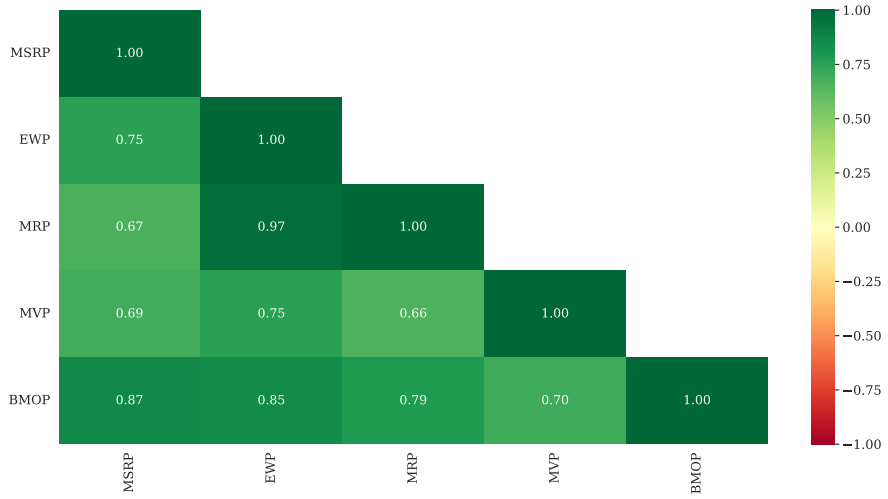

Fig. 6. Correlation of Daily Portfolio Returns. MSRP and MVP are less correlated to the other portoflios.

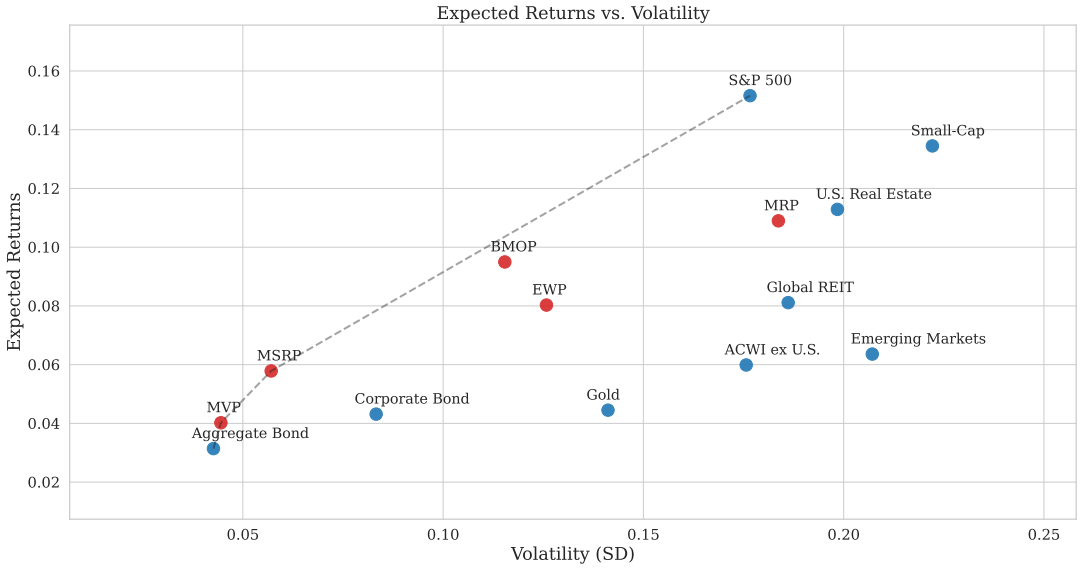

Fig. 7. Expected returns vs. Volatility for assets (in blue) and portfolios (in red). Each set of asset weights results in a portfolio than can be mapped into this two-dimensional space. The ones on the efficient frontiers (the dashed black line) are the portfolios with minimum volatility for a given target expected return.

necessary to create, evaluate, and verify portfolios, including data collection, optimization, backtesting, and visualization. Our sample investment scenario demonstrates that `PortFawn` can generate optimal portfolios for various targets. We release `PortFawn` code together with its documentation and highly encourage the reader to contribute to them.

References

[1] H. Markowitz, "Portfolio selection," 1959.

[2] R. J. Dowen, "Beta, non-systematic risk and portfolio selection," *Applied Economics*, vol. 20, no. 2, pp. 221–228, 1988.

[3] J. W. Cheng, "A switching regression approach to the stationarity of systematic and non-systematic risks: the hong kong experience," *Applied Financial Economics*, vol. 7, no. 1, pp. 45–57, 1997.

[4] B. Sofrankova, "Actual questions of risk management in models affecting enterprise performance," 2017.

[5] V. Todorov and T. Bollerslev, "Jumps and betas: A new framework for disentangling and estimating systematic risks," *Journal of Econometrics*, vol. 157, no. 2, pp. 220–235, 2010.

[6] R. Mansini, W. Ogryczak, and M. G. Speranza, "Twenty years of linear programming based portfolio optimization," *European Journal of Operational Research*, vol. 234, no. 2, pp. 518–535, 2014.

[7] K. Metaxiotis and K. Liagkouras, "Multiobjective evolutionary algorithms for portfolio management: A comprehensive literature review," *Expert Systems with Applications*, vol. 39, no. 14, pp. 11 685–11 698, 2012.

[8] A. Ponsich, A. L. Jaimes, and C. A. C. Coello, "A survey on multiobjective evolutionary algorithms for the solution of the portfolio optimization problem and other finance and economics applications," *IEEE Transactions on Evolutionary Computation*, vol. 17, no. 3, pp. 321–344, 2012.

[9] C. Aranha and H. Iba, "The memetic tree-based genetic algorithm and its application to portfolio optimization," *Memetic Computing*, vol. 1, no. 2, pp. 139–151, 2009.

[10] M. Corazza, G. Fasano, and R. Gusso, "Particle swarm optimization with non-smooth penalty reformulation, for a complex portfolio selection problem," *Applied Mathematics and Computation*, vol. 224, pp. 611–624, 2013.

[11] I. Strumberger, N. Bacanin, and M. Tuba, "Constrained portfolio optimization by hybridized bat algorithm," in *2016 7th International Conference on Intelligent Systems, Modelling and Simulation (ISMS)*. IEEE, 2016, pp. 83–88.

[12] S. K. Mishra, P. Ganapati, S. Meher, and R. Majhi, "A fast multiobjective evolutionary algorithm for finding wellspread pareto-optimal solutions," in *In KanGAL Report No. 2003002, lndian Institute Of Technology Kanpur*. Citeseer, 2002.

[13] R. Armananzas and J. A. Lozano, "A multiobjective approach to the portfolio optimization problem," in *2005 IEEE Congress on Evolutionary Computation*, vol. 2. IEEE, 2005, pp. 1388–1395.

[14] A. Baykasoğlu, M. G. Yunusoglu, and F. B. Özsoydan, "A grasp based solution approach to solve cardinality constrained portfolio optimization problems," *Computers & Industrial Engineering*, vol. 90, pp. 339–351, 2015.

[15] K. P. Anagnostopoulos and G. Mamanis, "The mean–variance cardinality constrained portfolio optimization problem: An experimental evaluation of five multiobjective evolutionary algorithms," *Expert Systems with Applications*, vol. 38, no. 11, pp. 14 208–14 217, 2011.

[16] J. Fliege and R. Werner, "Robust multiobjective optimization & applications in portfolio optimization," *European Journal of Operational Research*, vol. 234, no. 2, pp. 422–433, 2014.

[17] K. Lwin, R. Qu, and G. Kendall, "A learning-guided multi-objective evolutionary algorithm for constrained portfolio optimization," *Applied Soft Computing*, vol. 24, pp. 757–772, 2014.

[18] M. Shaverdi, S. Yaghoubi, and H. Ensafian, "A multi-objective robust possibilistic model for technology portfolio optimization considering social impact and different types of financing," *Applied Soft Computing*, vol. 86, p. 105892, 2020.

[19] F. Garcia, J. González-Bueno, F. Guijarro, J. Oliver, and R. Tamošiūnienė, "Multiobjective approach to portfolio optimization in the light of the credibility theory," *Technological and Economic Development of Economy*, vol. 26, no. 6, pp. 1165–1186, 2020.

[20] P. Krokhmal, J. Palmquist, and S. Uryasev, "Portfolio optimization with conditional value-at-risk objective and constraints," *Journal of risk*, vol. 4, pp. 43–68, 2002.

[21] Y. Crama and M. Schyns, "Simulated annealing for complex portfolio selection problems," *European Journal of operational research*, vol. 150, no. 3, pp. 546–571, 2003.

[22] H. Peng, G. Kitagawa, M. Gan, and X. Chen, "A new optimal portfolio selection strategy based on a quadratic form mean–variance model with transaction costs," *Optimal Control Applications and Methods*, vol. 32, no. 2, pp. 127–138, 2011.

[23] H. Soleimani, H. R. Golmakani, and M. H. Salimi, "Markowitz-based portfolio selection with minimum transaction lots, cardinality constraints and regarding sector capitalization using genetic algorithm," *Expert Systems with Applications*, vol. 36, no. 3, pp. 5058–5063, 2009.

[24] N. Suganya and G. V. Pai, "Constrained portfolio rebalancing with transaction costs using evolutionary wavelet hopfield network strategy," in *2009 World Congress on Nature & Biologically Inspired Computing (NaBIC)*. IEEE, 2009, pp. 451–456.

[25] S. S. Meghwani and M. Thakur, "Multi-criteria algorithms for portfolio optimization under practical constraints," *Swarm and evolutionary computation*, vol. 37, pp. 104–125, 2017.

[26] H. Babazadeh and A. Esfahanipour, "A novel multi period mean-var portfolio optimization model considering practical constraints and transaction cost," *Journal of Computational and Applied Mathematics*, vol. 361, pp. 313–342, 2019.

[27] T. Kadowaki and H. Nishimori, "Quantum annealing in the transverse ising model," *Physical Review E*, vol. 58, no. 5, p. 5355, 1998.

[28] G. E. Santoro, R. Martoňák, E. Tosatti, and R. Car, "Theory of quantum annealing of an ising spin glass," *Science*, vol. 295, no. 5564, pp. 2427–2430, 2002.

[29] S. Boixo, T. F. Rønnow, S. V. Isakov, Z. Wang, D. Wecker, D. A. Lidar, J. M. Martinis, and M. Troyer, "Evidence for quantum annealing with more than one hundred qubits," *Nature physics*, vol. 10, no. 3, pp. 218–224, 2014.

[30] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *science*, vol. 220, no. 4598, pp. 671–680, 1983.

[31] P. J. Van Laarhoven and E. H. Aarts, "Simulated annealing," in *Simulated annealing: Theory and applications*. Springer, 1987, pp. 7–15.

[32] A. B. Finnila, M. Gomez, C. Sebenik, C. Stenson, and J. D. Doll, "Quantum annealing: A new method for minimizing multidimensional functions," *Chemical physics letters*, vol. 219, no. 5-6, pp. 343–348, 1994.

[33] A. M. Childs, E. Farhi, and J. Preskill, "Robustness of adiabatic quantum computation," *Physical Review A*, vol. 65, no. 1, p. 012322, 2001.

[34] C. C. McGeoch, "Adiabatic quantum computation and quantum annealing: Theory and practice," *Synthesis Lectures on Quantum Computing*, vol. 5, no. 2, pp. 1–93, 2014.

[35] T. Albash and D. A. Lidar, "Adiabatic quantum computation," *Reviews of Modern Physics*, vol. 90, no. 1, p. 015002, 2018.

[36] G. G. Guerreschi and A. Y. Matsuura, "Qaoa for max-cut requires hundreds of qubits for quantum speed-up," *Scientific reports*, vol. 9, no. 1, pp. 1–7, 2019.

[37] S. Gharibian and O. Parekh, "Almost optimal classical approximation algorithms for a quantum generalization of max-cut," *arXiv preprint arXiv:1909.08846*, 2019.

[38] O. Titiloye and A. Crispin, "Quantum annealing of the graph coloring problem," *Discrete Optimization*, vol. 8, no. 2, pp. 376–384, 2011.

[39] K. Kudo, "Constrained quantum annealing of graph coloring," *Physical Review A*, vol. 98, no. 2, p. 022301, 2018.

[40] C. Silva, A. Aguiar, P. M. Lima, and I. Dutra, "Mapping graph coloring to quantum annealing," *Quantum Machine Intelligence*, vol. 2, no. 2, pp. 1–19, 2020.

[41] H. Ushijima-Mwesigwa, C. F. Negre, and S. M. Mniszewski, "Graph partitioning using quantum annealing on the d-wave system," in *Proceedings of the Second International Workshop on Post Moores Era Supercomputing*, 2017, pp. 22–29.

[42] C. F. Negre, H. Ushijima-Mwesigwa, and S. M. Mniszewski, "Detecting multiple communities using quantum annealing on the d-wave system," *Plos one*, vol. 15, no. 2, p. e0227538, 2020.

[43] S. Yarkoni, A. Plaat, and T. Back, "First results solving arbitrarily structured maximum independent set problems using quantum annealing," in *2018 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2018, pp. 1–6.

[44] B. Haddar, M. Khemakhem, S. Hanafi, and C. Wilbaut, "A hybrid quantum particle swarm optimization for the multidimensional knapsack problem," *Engineering Applications of Artificial Intelligence*, vol. 55, pp. 1–13, 2016.

[45] D. A. Battaglia, G. E. Santoro, and E. Tosatti, "Optimization by quantum annealing: Lessons from hard satisfiability problems," *Physical Review E*, vol. 71, no. 6, p. 066707, 2005.

[46] T.-J. Hsu, F. Jin, C. Seidel, F. Neukart, H. De Raedt, and K. Michielsen, "Quantum annealing with anneal path control: application to 2-sat problems with known energy landscapes," *arXiv preprint arXiv:1810.00194*, 2018.

[47] T. Krüger and W. Mauerer, "Quantum annealing-based software components: An experimental case study with sat solving," in *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops*, 2020, pp. 445–450.

[48] S. H. Adachi and M. P. Henderson, "Application of quantum annealing to training of deep neural networks," *arXiv preprint arXiv:1510.06356*, 2015.

[49] D. Crawford, A. Levit, N. Ghadermarzy, J. S. Oberoi, and P. Ronagh, "Reinforcement learning using quantum boltzmann machines," *arXiv preprint arXiv:1612.05695*, 2016.

[50] M. H. Amin, E. Andriyash, J. Rolfe, B. Kulchytskyy, and R. Melko, "Quantum boltzmann machine," *Physical Review X*, vol. 8, no. 2, p. 021050, 2018.

[51] D. Venturelli and A. Kondratyev, "Reverse quantum annealing approach to portfolio optimization problems," *Quantum Machine Intelligence*, vol. 1, no. 1, pp. 17–30, 2019.

[52] J. Cohen, A. Khan, and C. Alexander, "Portfolio optimization of 60 stocks using classical and quantum algorithms," *arXiv preprint arXiv:2008.08669*, 2020.

[53] E. Grant, T. S. Humble, and B. Stump, "Benchmarking quantum annealing controls with portfolio optimization," *Physical Review Applied*, vol. 15, no. 1, p. 014012, 2021.